# Instructions

RXY and RI formats

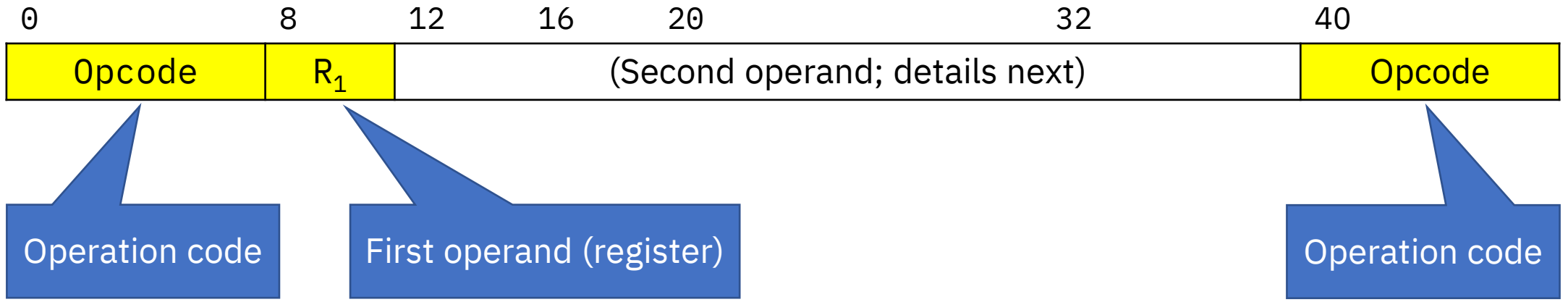
# Instruction formats

- Next, RXY instructions

| Format | Description |
|---|---|
| Register-and-register (**RR**) | Both operands are registers |
| Register-and-storage (**RS**)<br>Register-and-indexed-storage (**RX**) | One operand is a register<br>and another a storage address |
| Storage-and-immediate (**SI**) | One operand is a storage address<br>and another an “immediate” value |
| Storage-and-storage (**SS**) | Both operands are storage addresses |
| Register-and-storage (**RXY**) | One operand is a register<br>and another a long displacement address |
| Register-and-immediate (**RI**) | One operand is a register<br>and another a 16-bit relative address |

# RXY instructions

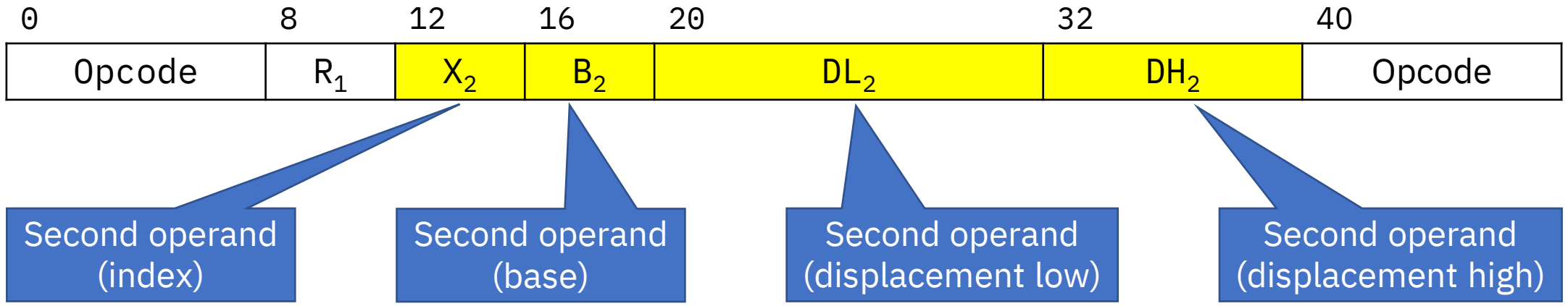
- RXY instructions occupy 3 halfwords
- Their structure in memory is slightly more complicated, so we'll see it in two stages:




- Note the 16-bit *extended opcode:* 8 bits in the first byte and 8 bits in the last one

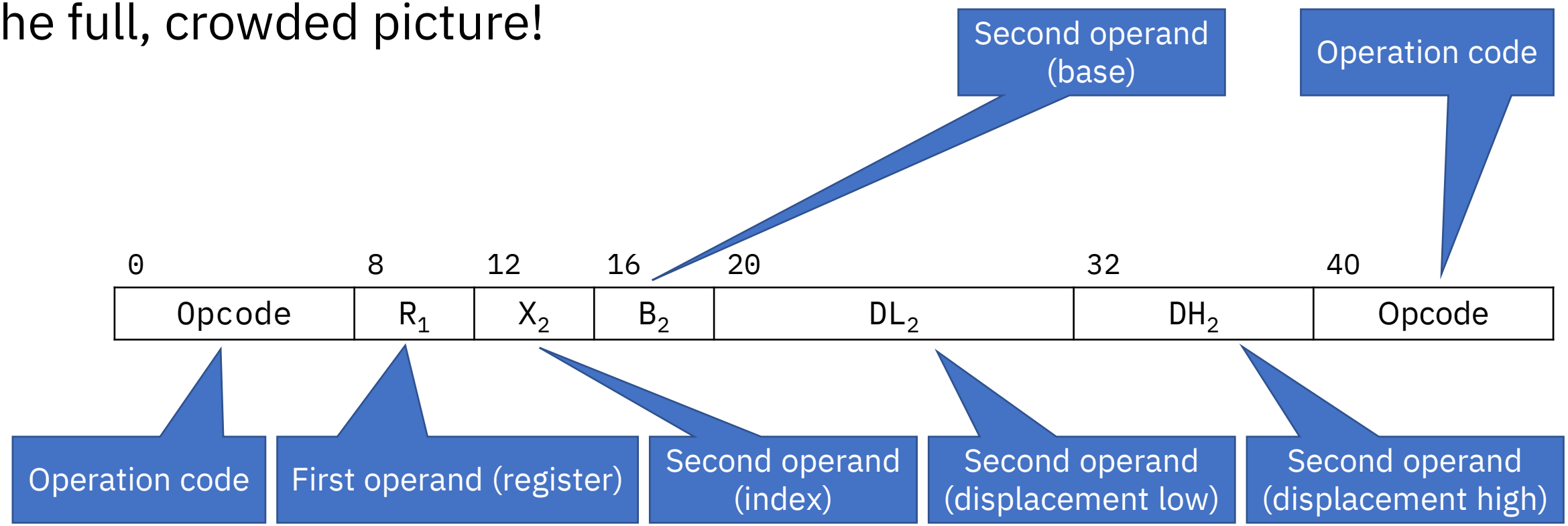
# RXY instructions

- The second operand is a long displacement address:



# RXY instructions

- The full, crowded picture!

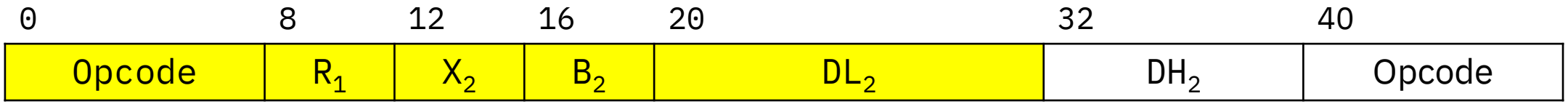


# RXY instructions

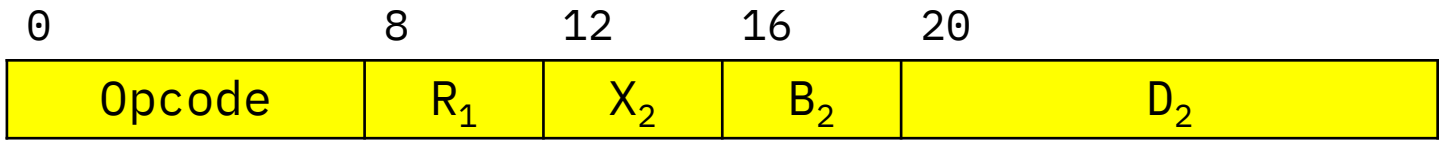
- Apart from having a long displacement operand, RXY instructions behave just like RX instructions

- In fact, the first four bytes of RX instructions have the same format as the first four bytes of RXY instructions:

- RXY:




- RX:



# RXY instructions - example

- We’ve seen the ADD instruction in RR and RX formats
- There’s an RXY format of ADD
- Mnemonic: **AY**
  - Note the “Y” in the mnemonic, it indicates long displacement
- Opcode: **E35A**
  - Note the (2-byte) extended opcode

# RXY instructions - ADD example

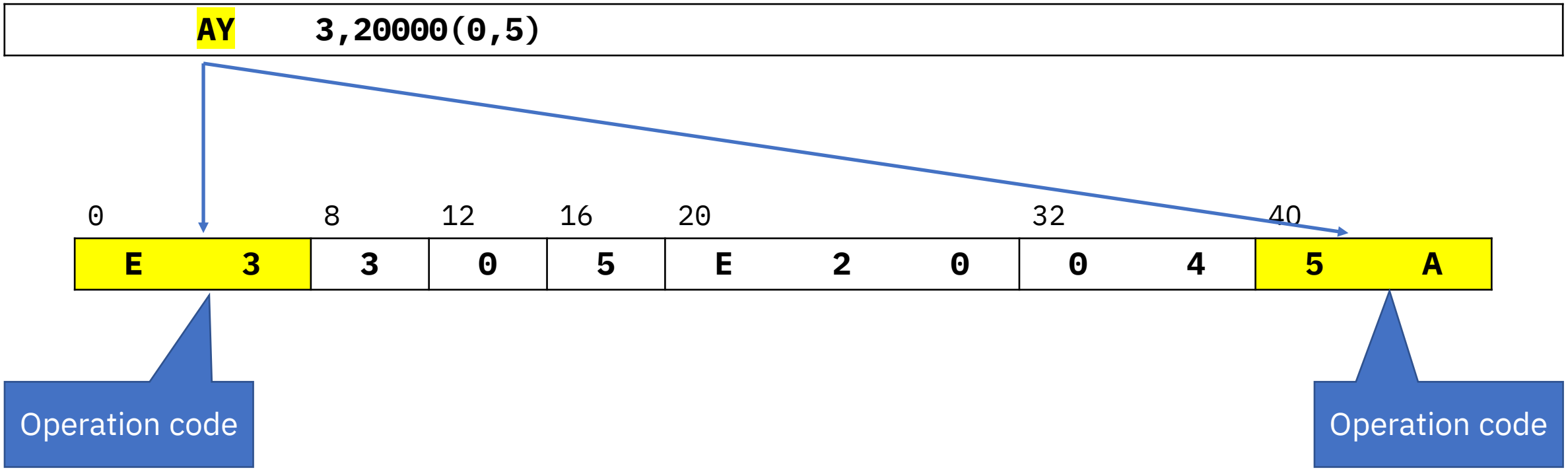
- “ADD to register 3 the fullword at displacement 20000 from register 5”
- Note we can’t use the RX-type ADD, because the displacement exceeds 4095 bytes
- The Assembler source statement is very similar to the RX example:

```
        AY    3,20000(0,5)
```

# RXY instructions - ADD example

- Decimal 20000 is hex **04E20**

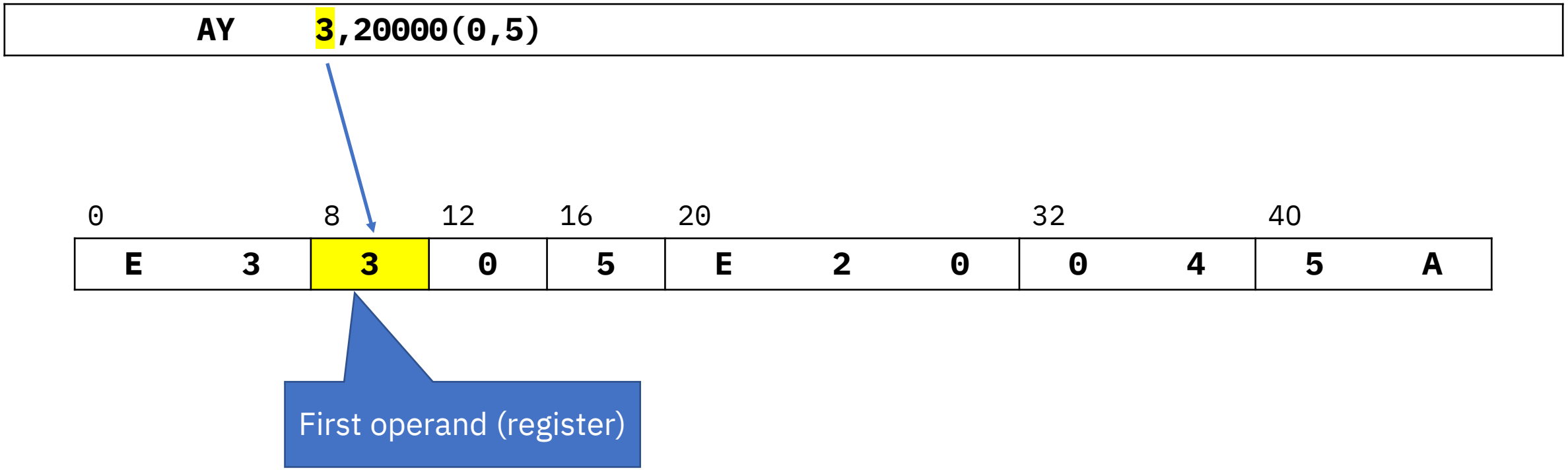
- The instruction, in memory, is:



# RXY instructions - ADD example

- Decimal 20000 is hex **04E20**

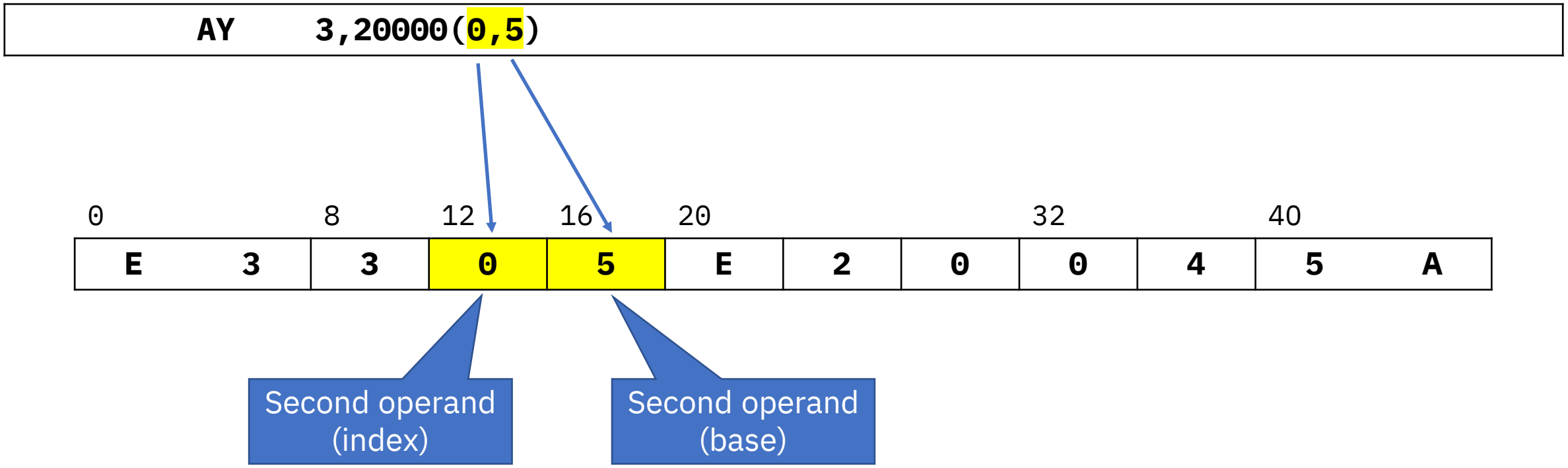
- The instruction, in memory, is:



# RXY instructions - ADD example

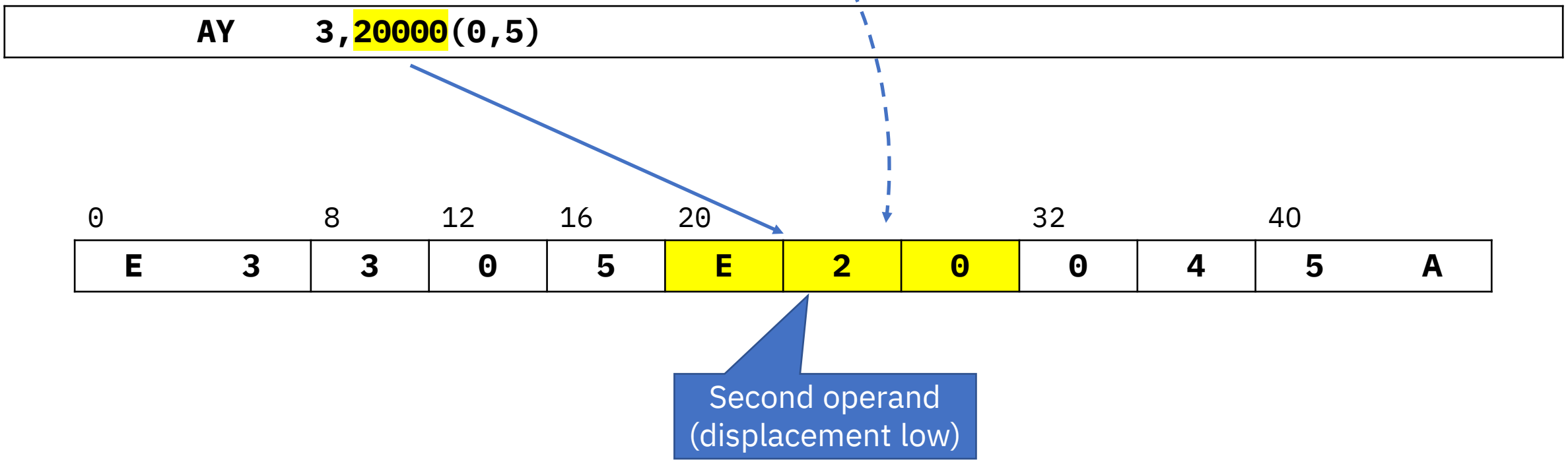
- Decimal 20000 is hex **04E20**

- The instruction, in memory, is:



# RXY instructions - ADD example

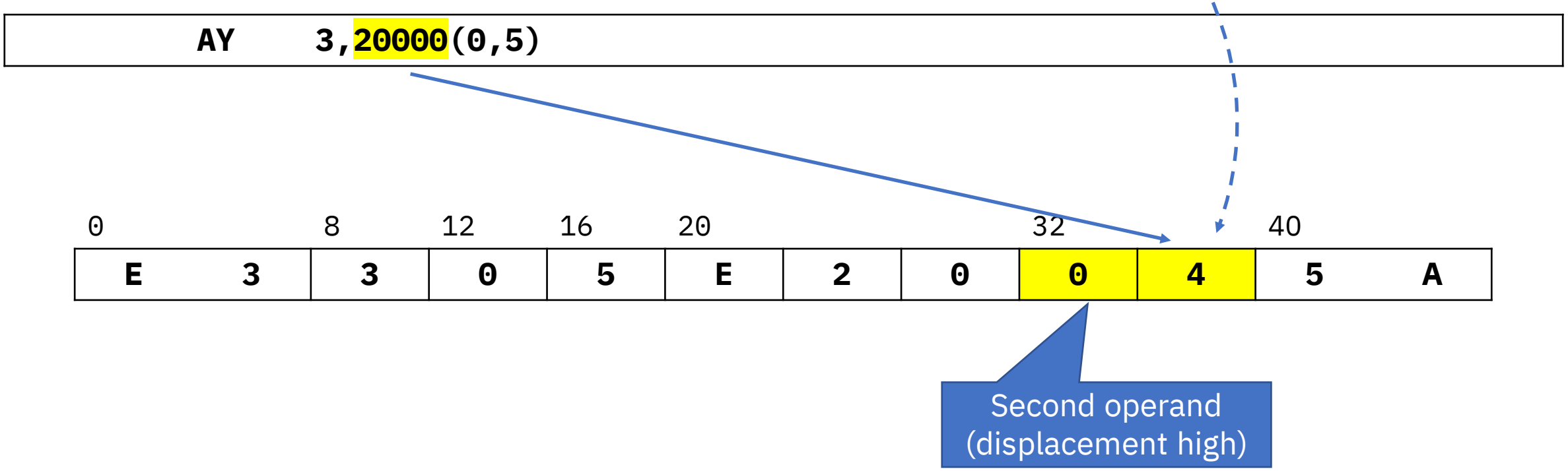
- Decimal 20000 is X'**04E20**'
- The instruction, in memory, is:



# RXY instructions - ADD example

- Decimal 20000 is hex **04E20**

- The instruction, in memory, is:



# Instruction formats

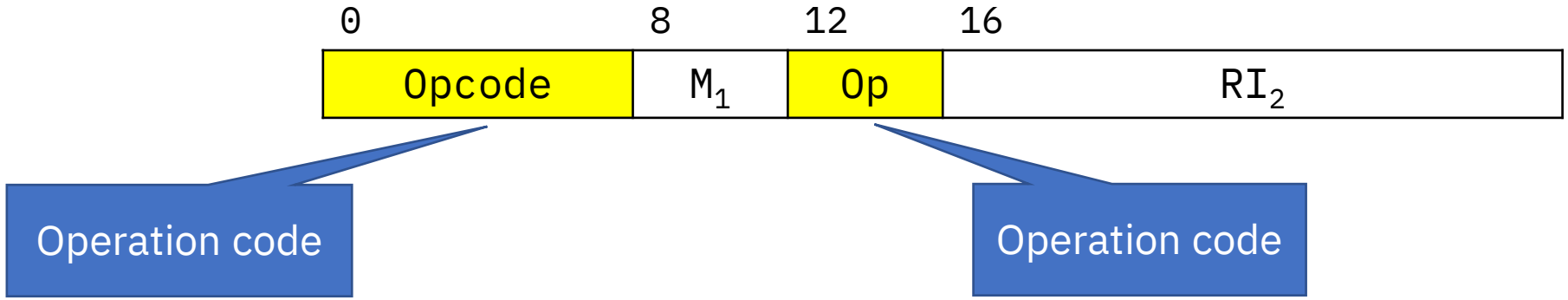
- Next, RI instructions

| Format | Description |
|---|---|
| Register-and-register (**RR**) | Both operands are registers |
| Register-and-storage (**RS**)<br>Register-and-indexed-storage (**RX**) | One operand is a register<br>and another a storage address |
| Storage-and-immediate (**SI**) | One operand is a storage address<br>and another an “immediate” value |
| Storage-and-storage (**SS**) | Both operands are storage addresses |
| Register-and-storage (**RXY**) | One operand is a register<br>and another a long displacement address |
| Register-and-immediate (**RI**) | One operand is a register<br>and another a 16-bit relative address |

# RI instructions

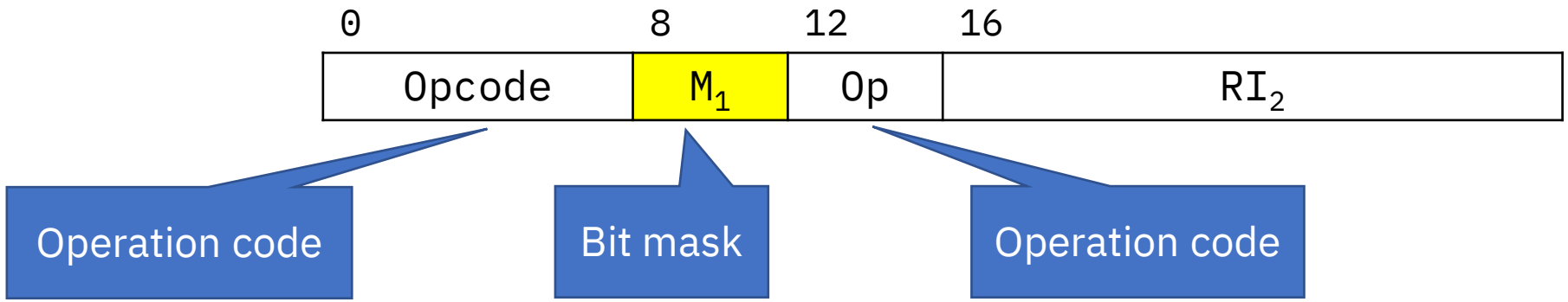
- RI instructions occupy 2 halfwords
- In RI instructions, the second operand is an immediate value
- There are three variants of RI instructions; we'll see one where the immediate value is a relative address
  - We'll see the other two variants as needed
- Next, the instruction format

# RI instructions



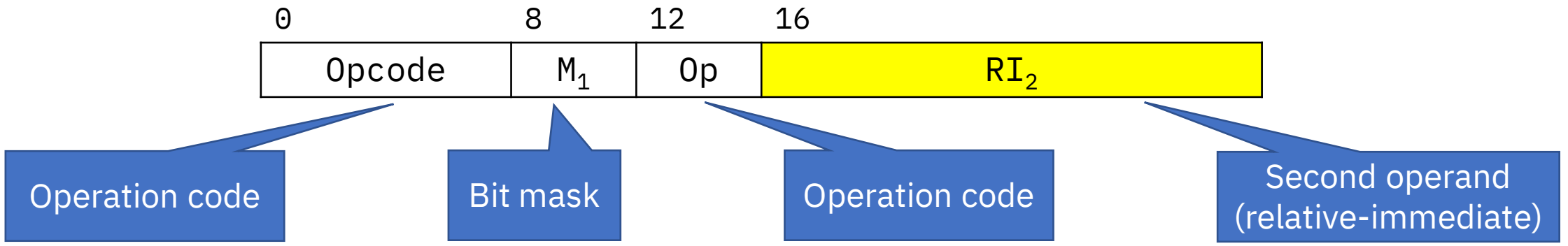

- RI instructions have an extended (12-bit) operation code

# RI instructions




- RI instructions have an extended 12-bit operation code
- A 4-bit mask controls the instruction operation (details later)

# RI instructions




- RI instructions have an extended 12-bit operation code
- A 4-bit mask controls the instruction operation (details later)
- As you know, the second operand is a signed displacement: number of halfwords from the instruction itself

# RI instructions - examples

- We saw the **JUMP** instruction in the last unit
- It is a form of Branch that uses relative addressing instead of base-displacement
- Mnemonic: **J**
- Opcode: **A74**
  - Note the (12-bit) extended opcode

# RI instructions - JUMP example

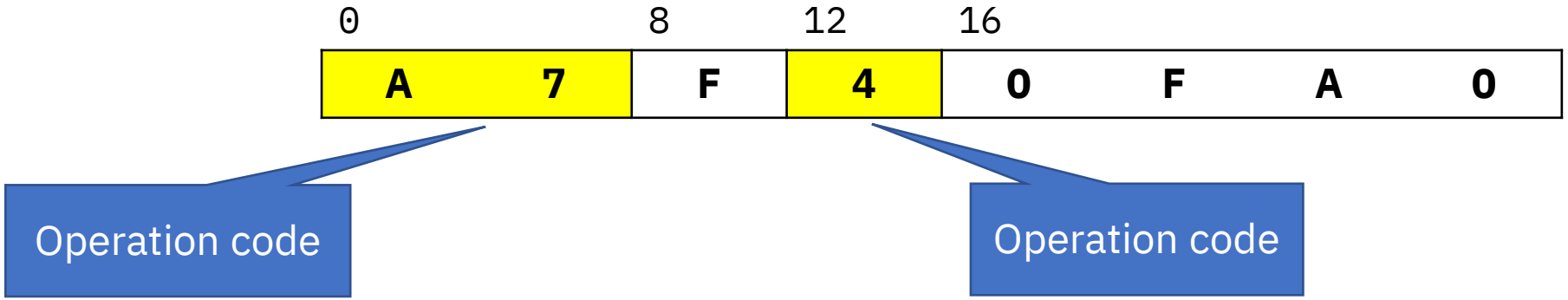
- “JUMP to the instruction at 8000 bytes from this one”
- The Assembler source statement is:

```
        J       *+8000
```

- Note the asterisk: it means “here”, or “this location”

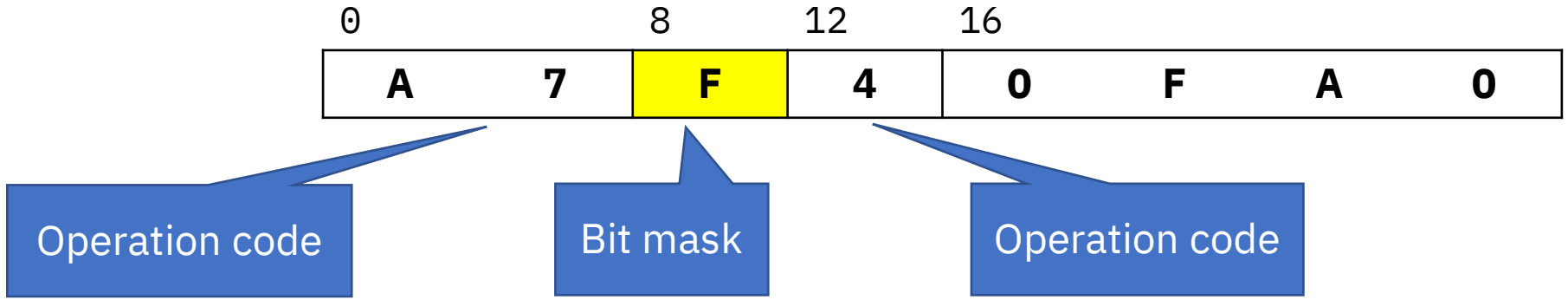
# RI instructions - JUMP example

- The instruction, in memory, is:



# RI instructions - JUMP example

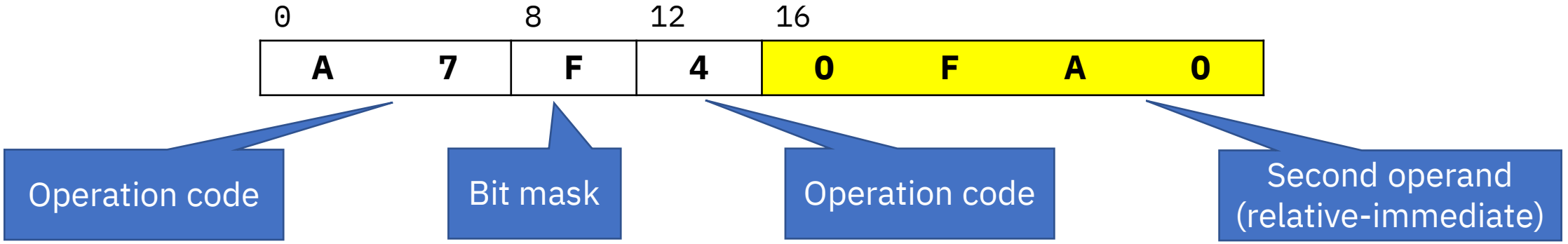
- The instruction, in memory, is:




- The mask control whether the jump occurs
  - **X'F'** means “always jump”
  - We’ll learn more when we study Branching in more detail

# RI instructions - JUMP example

- The instruction, in memory, is:




- The mask control whether the jump occurs
  - Hex F means “always jump”
  - We’ll learn more when we study Branching in more detail

- 8000 bytes is 4000 halfwords. 4000 is **X’0FA0’**

# Summary

- This concludes the introduction to the elements of the z/Architecture that an Assembler application programmer deals with
- We don’t yet have the full picture; other elements will be added as needed
- Time to start learning the Assembler language syntax!